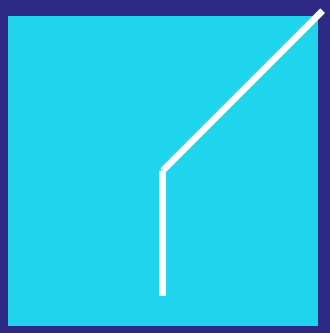
ENGINEERING REPORT / 2026

# FlakyScan

A practical .NET CLI for finding flaky tests

PROJECT OVERVIEW



## Run. Compare. Understand.

A small, focused tool that turns repeated test runs into a clear Markdown report developers can act on.

Built with .NET 8 ? TRX + System.Xml.Linq ? xUnit sample

01 / PRODUCT SHAPE

# What we built

A deliberately small surface area, with each responsibility easy to find and test.

RUN

Repeat the same test command

TestRunner

READ

Understand TRX results safely

TrxParser

DECIDE

Detect mixed outcomes

FlakyDetector

## The user experience

The command has one central promise: show which tests changed behavior across runs.

That promise shaped the implementation and kept unrelated features out of scope.

INPUT

--path <project-or-solution>
--runs <positive number>

Simple arguments

OUTPUT

Markdown table
exit code 0 or 1

Terminal-friendly

## Included in the repository

- Console application targeting .NET 8
- Four small production classes
- Parser and detector unit tests
- Four-test sample project: two stable, two intentionally flaky
- GitHub Actions workflow for push and pull request validation

# How a scan works

The pipeline is linear on purpose: run each time, parse each result, then compare outcomes.

## The scan pipeline

01
### CLI
Parse options and validate path

02
### RUN
dotnet test N times

03
### PARSE
Read TRX with LINQ to XML

04
### DETECT
Group by test and compare

## The result model

```
TestResult

TestName    = fully qualified name Outcome     = Passed | Failed | Ignored  FlakyTest
Passed      = count of pass outcomes Failed    = count of fail outcomes FailureRate  = Failed / (Passed + Failed)
```

## Why skipped tests are ignored

A skipped or not-executed test did not provide evidence of either success or failure.

Keeping it out of the denominator avoids turning an incomplete run into a false signal.

# From outcomes to signal

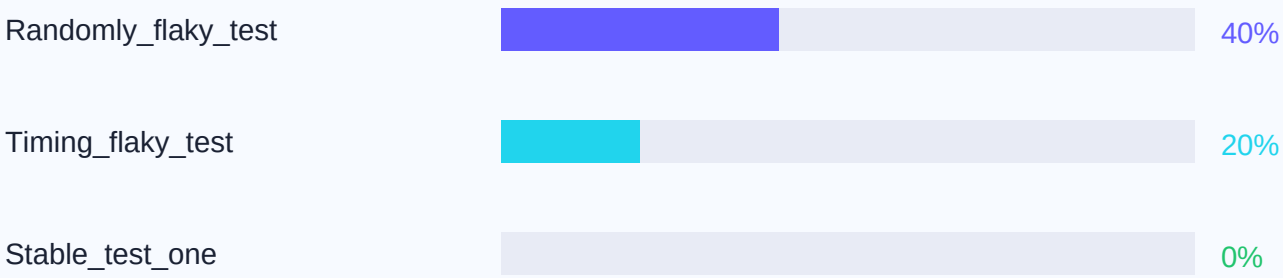
A flaky test is not merely a failure; it is a test whose result changes across runs.

FLAKY = at least one pass + at least one failure

Skipped outcomes are excluded before the comparison.

## Example: five runs of the sample project

Each bar shows the proportion of runs that failed for a test.




## Report output

| Test name | Passed | Failed | Failure rate |
|---|---|---|---|
| Randomly_flaky_test | 3 | 2 | 40.0% |
| Timing_flaky_test | 4 | 1 | 20.0% |

Results are sorted by failure rate descending, so the strongest signal appears first.

# Verification, usage, and boundaries

The first version is intentionally complete for its job, without pretending to solve every testing problem.

## What we verified

| 6 | 0 | 4 | 1 |
|---|---|---|---|
| unit tests passing | build warnings | sample tests | clear CLI job |

## Try it

COMMAND

```
dotnet run --project src/FlakyScan --
  --path ./samples/FlakySample --runs 5
```

Add --fail-on-flaky when a CI job should fail if any mixed outcome is found.

## Kept intentionally out of scope

- HTML or JSON reports
- Failure-pattern analysis
- Parallel test runs
- Configuration files
- dotnet tool packaging

The result is a small codebase that is straightforward to review, run, and extend later.